



[BIT703 Web Technologies](#) / [Contents](#) / Topic 7: Client-side web application development and programming – JavaScript / Understanding JavaScript

JavaScript async and await 25m

JavaScript runs code in a non-blocking way. This means that because some code takes time to execute, JavaScript will execute that code in the background while it continues to execute other code in parallel.

To understand non-blocking, consider a checkout person in a supermarket who is waiting for their customer to count their cash or transfer funds. While they do this, the checkout person can serve the other customers waiting in the queue. When the initial customer has finished sorting out their stuff, the checkout person will be able to serve them.

Because JavaScript works in a non-blocking way, additional measures must be taken if your code needs to have a result available before further code is executed. This is where `async` and `await` come into play. It is quite common to use `async` and `await` in web applications (such as social media applications) that need to wait before responding.

Async function

Async means asynchronous. It allows a program to run a function without freezing the entire program. This is done using the `async` and `await` keywords.

Watch *What is asynchronous JavaScript code?* (Bro Code, 2023) (4:47), which introduces the concept of asynchronous JavaScript.



Skip to main content

ENTER FULLSCREEN

What is asynchronous JavaScript code?



What is asynchronous JavaScript code? ^{zz}

Bro Code



Watch on

Search...



Yoh. What's going on, people. So in today's video, I gotta explain what asynchronous code is in JavaScript. Synchronous is code that executes line by line consecutively in a sequential manner. Synchronous code waits for an operation to complete. For example, if I were to use `console.log`-- let's pretend we're performing some task. It doesn't matter what the task is, we will pretend that task 1 is complete. Then we'll move on to task 2, then task 3.

All this code is synchronous. We're executing this code line by line. We can't move on to task 2 until we complete task 1. There is an order of events that we need to follow. It's code that waits for an operation to complete. Now, on the other hand, asynchronous code that allows multiple operations to be performed concurrently without waiting. Asynchronous code doesn't block the execution flow and allows the programme to

Powered by 3PLAYMEDIA



[Skip to main content](#)

Await function

The `await` keyword is used inside the `async` function to wait for the asynchronous operation.

The `await` syntax is:

```
<code><b>let</b> result = <b>await</b> promise;</code>
```

The use of `await` pauses the `async` function until the Promise (discussed below) returns a result (resolve or reject) value.

How `async` and `await` work in JavaScript

`Async` and `await` make it easier to write Promises. The keyword `async` before a function makes the function return a Promise, always. The keyword `await` is used inside `async` functions, which makes the program wait until the Promise resolves.

`async` makes a function return a Promise.

`await` makes a function wait for a Promise.

Watch *JavaScript `async/await` is easy!* (Bro Code, 2023) (4:38), where the simplicity and power of `async/await` are explained, demonstrating how they make working with asynchronous JavaScript code more readable and intuitive.



ENTER FULLSCREEN

[Skip to main content](#)

JavaScript ASYNC/AWAIT is easy!

JavaScript ASYNC/AWAIT is easy! ⌚
Bro Code



Watch on

Search...



No transcript available.

Powered by 3PLAYMEDIA



[Skip to main content](#)

How Promises work in JavaScript

A Promise is a JavaScript object that may produce a value at the later part of a program execution. The value produced can either be resolved (for example, a result value) or unresolved (for example, an error like a network failure). It works like a real-life promise: it does not immediately return a value but promises to return a value in the future.

A Promise can be in one of three states:

- **Fulfilled:** meaning the operation was completed successfully (for example, `resolve()` was called)
- **Rejected:** meaning the operation has failed (for example, `reject()` was called)
- **Pending:** meaning the operation is not yet fulfilled or rejected.

A pending Promise can either be fulfilled with a value or rejected with a reason (error).

Using a Promise is shown below:

```
myFunction().then(  
  
function(value) { /* code if successful */ },  
  
function(error) { /* code if some error */ }  
  
);
```

You can use the code `Promise (function(resolve, reject))`. This has two arguments – `resolve` and `reject` – which are predefined by JavaScript. Often there is no need to use the `reject` function.

Watch *Async/await explained by doing your morning routine* (CodeAnalogies Tutorial 2020) (4:14) to understand how `async`, `await` and a promise work. This video demonstrates how you use these functions by using an example of a morning routine that involves five morning tasks.



ENTER FULLSCREEN

[Skip to main content](#)

Async/Await Explained By Doing Your Morning Routine

Async/Await Explained By Doing Your Morning Routine

CodeAnalogies Tutorials



Watch on



No transcript available.

Powered by  3PLAYMEDIA



[Skip to main content](#)

HTMLCSSJSResult

EDIT ON

LIVE

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet"
type="text/css" href="styles.css">
</head>
<body>

<h2>JavaScript async / await</h2>
<p id="demo"></p>

<script src="script.js"></script>

</body>
</html>
```

Resources1x0.5x0.25xRerun

Example 7.3 Async and await

☒ Async and await

Non-Assessed

[Skip to main content](#)

Identify the part of the code that has been executed in Example 7.3.

```
function myDisplayer(some) {  
  
    document.getElementById("demo").innerHTML = some;  
  
}  
  
async function myFunction() {  
  
    return "Hello BIT703";  
  
}  
  
myFunction().then(  
  
    function(value) { myDisplayer(value); },  
  
    function(error) { myDisplayer(error); }  
  
);
```

Submit



☒ Promise

Non-Assessed

[Skip to main content](#)

Copy the code below and run it in a code editor such as CodePen, then answer the questions below.

```
<!DOCTYPE html>
<html>
  <body>
    <h2>JavaScript async / await</h2>
    <h1 id="demo"></h1>
    <script>
      async function myDisplay() {
        let myPromise = new Promise(function (resolve, reject) {
          resolve("I love Javascript !!");
        });
        document.getElementById("demo").innerHTML = await myPromise;
      }
      myDisplay();
    </script>
  </body>
</html>
```

1. Why has the await command been used in the above code?
2. Modify the code as directed below:
 - replace the code: `let myPromise = new Promise(function(resolve, reject)`
with the code: `let myPromise = new Promise(function(resolve)`
 - will the code execute using this new code? Explain why or why not.

B	<i>I</i>	<u>U</u>		☰	¹ / ₂ ☰	☰☰	☰☰☰	☰☰☰☰		T ²	T ₂		☐☐☐☐
													0 / 50 Word Limit

[Save draft](#)[Submit](#)

[Skip to main content](#)

Dig deeper

To learn more about async and await functions and to see more examples of these functions, head to Async Function (MDN, 2025).



Thoughts on this page?

